

1. Anatomia do Ficheiro Twee 3 (Os Vértices)

O formato Twee funciona como o esqueleto do grafo. A sua função é definir onde começa e acaba cada nó (Cena).

A Passagem (Nó)

Cada cena no **Plot in a Pot** corresponde a uma "passagem". No ficheiro `.twee`, um nó é definido por um cabeçalho que utiliza o delimitador `::`.

- **Identificador (ID):** O nome que segue os `::` é o ID único do vértice na lista de adjacência.
- **Metadados:** Informação entre chavetas `{ }` que o nosso editor usa para guardar a posição `$(x, y)$` do nó no ecrã.
- **Etiquetas (Tags):** Palavras entre parênteses retos `[ ]` que podemos usar para filtrar tipos de cenas (ex: `[combate]`, `[final]`).

Exemplo de Código:

Fragmento do código

```
:: Praca_Central {"position":"100,200"} [cidade]
```

```
Estás na praça principal. O sol brilha intensamente.
```

2. O Modelo SugarCube (As Arestas e a Fita de Dados)

Enquanto o Twee define os pontos, o SugarCube define como navegamos entre eles e como o estado do jogo muda. Para o nosso motor em Python, o *parser* deve focar-se em dois elementos:

2.1. Criação de Arestas (Links)

As arestas do grafo são extraídas do texto entre parênteses retos duplos `[ [ ] ]`.

- **Aresta Simples:** `[[Ir para a Taverna]]` -> Cria uma ligação para o nó "Ir para a Taverna".
- **Aresta com Alias:** `[[Abrir a porta|Cena_Interior]]` -> O utilizador vê "Abrir a porta", mas o ponteiro do grafo aponta para o ID "Cena_Interior".

2.2. Manipulação da Máquina de Estados (Macros)

O SugarCube utiliza macros para gerir a "fita de dados" que corre em paralelo com o grafo.

- **Set (Alteração de Estado):** `<<set $ouro += 10>>` -> O nosso motor deve intercetar isto para atualizar as variáveis globais.
- **If (Condicional de Aresta):** Permite que uma escolha só apareça se certas condições forem metidas.
- Fragmento do código
- List
- `<<if $tem_chave>>`
- `[[Abrir Baú|Cena_Tesouro]]`
- `<</if>>`

3. Integração com a Lista de Adjacência Bidirecional

Para o **Plot in a Pot**, o processo de leitura do ficheiro deve seguir esta lógica algorítmica:

1. **Fase de Scan:** O programa percorre o ficheiro à procura de `:`. Cada vez que encontra um, cria um novo objeto `Cena` na `lista`.
2. **Fase de Linkage (Sucessores):** Dentro de cada cena, o programa procura por `[[ ]]`. Para cada link encontrado, adiciona o destino à lista de **Saídas** do nó atual.

Fase de Retrocesso (Predecessores): Por cada Saída registada, o motor vai ao nó de destino e adiciona o ID do nó atual à sua lista de **Entradas**. Isto **garante** que, na nossa interface, possas sempre clicar numa cena e ver todos os caminhos que levam até ela.

Especificação Twee 3 (v3.0.2)

1. Introdução e Propósito

⋮ O que diz a especificação:

O Twee é um formato de texto utilizado para estruturar o código-fonte de histórias criadas no Twine. Funciona como um equivalente legível por humanos dos ficheiros HTML gerados pelo Twine (que contêm os dados da história e das passagens). Com a evolução do software (do Twine 1 para o Twine 2), o Twee tornou-se um formato independente, permitindo que outros compiladores o utilizassem para gerar histórias. Esta especificação (v3) serve para unificar as várias extensões criadas pela comunidade num único padrão oficial.

Explicação Prática:

O Twee é uma forma de representar um Grafo Direcionado $G = (V, E)$ num ficheiro de texto simples (.txt, essencialmente). Em vez de utilizar uma estrutura estritamente hierárquica como o JSON (que pode ser difícil de escrever à mão em histórias longas), o Twee usa marcações simples para separar os vértices V (chamados aqui de "Passagens").

2. Tipo de Ficheiro e Notação Base

O que diz a especificação:

Recomenda-se a utilização das extensões .tw ou .twee e a codificação UTF-8. Um ficheiro Twee é composto por uma ou mais "passagens" (passages). Cada passagem divide-se em duas partes: um Cabeçalho (Header) e o Conteúdo (Content).

O Cabeçalho da Passagem (Passage Header)

O que diz a especificação:

O cabeçalho tem de ocupar apenas uma linha e seguir esta ordem rigorosa:

1. Símbolo de início obrigatório: dois pontos duplos (::) no começo da linha.
2. Nome da passagem (obrigatório).
3. Bloco de *tags* (etiquetas) opcional, que deve seguir imediatamente o nome.
4. Bloco de metadados opcional (um bocado de JSON na mesma linha).

Para evitar erros de leitura pelo compilador, se o nome da passagem incluir os caracteres especiais usados nos blocos ([,], {, }), estes devem ser escapados utilizando uma barra invertida (\). O mesmo aplica-se à própria barra invertida (escrita como \\).

Exemplos de Sintaxe:

Fragmento do código

```
:: Um caminho coberto de mato  
:: Um caminho coberto de mato [floresta assustador]  
:: Um caminho coberto de mato {"position":"600,400","size":"100,200"}  
:: Um caminho coberto de mato [floresta assustador] {"position":"600,400","size":"100,200"}
```

Explicação Prática:

O cabeçalho atua como o identificador primário (ID) na Lista de Adjacência. O símbolo :: funciona como um delimitador que indica ao software: "Aqui começa um novo nó da história".

Regras dos Blocos do Cabeçalho:

- **Nome:** Tem de ser único no projeto. Recomenda-se não usar caracteres de hiperligação ([,], |).

O compilador deve emitir um aviso se encontrar nomes duplicados.

- **Tags:** Uma lista de palavras separadas por espaços dentro de parênteses retos []. As *tags* não podem conter espaços. Servem para categorizar cenas (ex: marcar um nó como um evento de combate ou de diálogo).
- **Metadados (JSON):** Utilizado principalmente para a interface gráfica do editor. Suporta as chaves "position" (coordenadas X, Y da caixa no ecrã) e "size" (largura e altura). Se o JSON for inválido, o compilador ignora-o e continua.

O Conteúdo da Passagem (Passage Content)

O que diz a especificação:

O conteúdo começa na linha imediatamente a seguir ao cabeçalho e continua até o compilador encontrar o próximo cabeçalho (::) ou o fim do ficheiro. Linhas em branco no final do conteúdo devem ser ignoradas.

Explicação Prática:

É aqui que reside o texto narrativo e a lógica de transição (as arestas do grafo). Tudo o que for escrito aqui pertence ao nó definido no cabeçalho anterior.

Caso por algum motivo alguma linha do texto narrativo comece com :: o twine adiciona automaticamente um \ antes

```
\:: teste  
aoshdasljhd
```

3. Passagens Especiais (Special Passages)

O que diz a especificação:

Existem nomes de passagens reservados que têm funções específicas e não representam cenas da história.

- :: StoryTitle: Define o nome do projeto.
- :: StoryData: Um bloco JSON que encapsula as configurações globais do projeto.
 - ifid: (Obrigatório) Identificador de Ficção Interativa. Um código UUID único para garantir que não há conflitos entre projetos diferentes no mundo.
 - format e format-version: O formato de compilação pretendido (ex: SugarCube, Harlowe).
 - start: Define qual é o nó inicial (\$v_0\$) por onde a história começa.
 - tag-colors: Permite associar cores às *tags* no editor visual.
 - zoom: O nível de zoom guardado no editor visual.
- :: Start: A passagem inicial por defeito, caso o nó inicial não esteja explicitamente definido no bloco StoryData.

Explicação Prática:

Estas passagens funcionam como o ficheiro de configuração global do software. Em vez de ter um ficheiro separado para as configurações

Heading 1

```
PS E:\DEV\PiaP> ollama run gemma4:e4b
```

pulling manifest

pulling 4c27e0f5b5ad: 100%

[illegible]

9.6 GB

pulling 7339fa418c9a: 100%

[illegible]

11 KB

pulling 56380ca2ab89: 100%

[illegible]

42 B

pulling f0988ff50a24: 100%

[illegible]

473 B

```
verifying sha256 digest
```

writing manifest

success

>>> consegues criar uma história curta não linear (com escolhas)

Thinking...

Here's a thinking process that leads to the suggested story framework:

1. ****Analyze the Request:**** The user wants a "short, non-linear story with choices" (uma história curta não linear (com escolhas)).
2. ****Define "Non-Linear" and "Choices":****
 - * ***Non-Linear:** The narrative doesn't follow a strict A $\rightarrow$ B $\rightarrow$ C sequence. Events might jump backward, skip ahead, or present branching paths that could revisit concepts or time periods.
 - * ***Choices:** The user must make decisions that affect the plot, character development, or outcome.
3. ****Determine the Medium/Format:**** Since I am text-based, I cannot create an interactive video game. I must create a **framework** that guides the user through the choices, making the experience feel immersive and customizable.
4. ****Establish a Core Premise/Setting (The Hook):**** The setting needs to be mysterious, flexible, and allow for multiple paths. A time loop, a memory puzzle, or a specific, limited location (like an old house, a station, or a strange forest) works best.
 - * ***Choice:** A memory puzzle/mystery involving a strange artifact and a sense of disorientation is ideal. Let's go with an old, forgotten space: a clock tower/archive.
5. ****Develop the Structure (The Rules of Engagement):****
 - * ***Introduction:** Set the scene and establish the central mystery.
 - * ***First Choice (The Inciting Incident):** Give the user a decision that fundamentally changes their immediate path.
 - * ***Second Choice (The Deepening Mystery):** Present information and require a risk/bet.

* *Third Choice (The Climax/Resolution):* Lead to a point where the user must use what they've learned to resolve the core conflict, resulting in different endings/outcomes.

6. **Drafting the Narrative (Self-Correction during drafting):**

* *Initial thought:* Just give them random choices. *Correction:* No, the choices must feel *consequential* and tied to the narrative logic.

* *Focus on Sensory Detail:* Use evocative language (dust, ticking, faint smell) to draw the reader in.

* *Structuring the Flow:* I will write the first segment and wait for the user's response before continuing the story. This is crucial for the "interactive" feel.

7. **Final Review:** Does it meet the criteria?

* *Short:* Yes, the overall experience is self-contained.

* *Non-linear:* Yes, the paths diverge and recombine based on choices.

* *With choices:* Yes, explicit decisions are required.

(The resulting Portuguese response is generated, focusing on clear instructions and an engaging opening scene.)
...done thinking.

Sim, consigo.

Como este é um formato de texto, vou criar a história em partes, e a sua resposta (a escolha que fizer) determinará qual será o caminho que a narrativa vai seguir.

Prepare-se para se perder.

O Relógio do Esquecimento

(O Início)

Você acorda em um lugar que não deveria existir.

Não há janela, nem porta visível. Apenas a iluminação crepuscular vinda de uma única fonte indistinta e o som constante, hipnótico, de um relógio. Não é um único relógio, mas uma sinfonia de tic-tacs desiguais, como se o tempo estivesse quebrado em centenas de pedaços.

Está em uma vasta biblioteca circular, repleta de estantes de madeira escura que se perdem na névoa. As prateleiras não contêm livros comuns; os volumes parecem feitos de materiais estranhos — pergaminhos de metal, placas de cristal, e livros cujas páginas brilham com um calor fraco, como brasas.

No centro da câmara, há uma mesa de carvalho e sobre ela, um único objeto: **um relógio de pêndulo parado**. Ele não marca hora, mas sim datas em sistemas de numeração que você nunca viu. Abaixo dele, há um caderno de capa de couro, aberto em uma página manchada, que contém apenas

1. Plot in a Pot

Até agora, a nossa base teórica focava-se em Grafos Direcionados Acíclicos (DAGs). No entanto, percebemos que para criar um software de ajuda à escrita de narrativas não lineares verdadeiramente flexível, impor a restrição matemática de ausência de ciclos acaba por ser uma limitação.

Ao abandonar a obrigatoriedade dos DAGs, o projeto passa a assentar em Grafos Direcionados gerais, o que significa que os ciclos narrativos passam a ser permitidos.

Pontos Positivos da Mudança

- Estruturas "Hub" e Regressos: Permite-me criar mecânicas narrativas onde a personagem volta a uma praça central ou a um menu principal após explorar diferentes ramos.
- Mecânicas de Simulação e Tempo: Como notei na nossa pesquisa sobre a Máquina de Estados, torna-se possível implementar ações de "Espera". Isto gera uma ligação que aponta da cena para si mesma (um loop fechado), avançando as variáveis de tempo na fita de estado sem mudar a localização do jogador.
- Maior Liberdade Orgânica: Não obrigo o escritor a pensar num fluxo estritamente sequencial e unidirecional.

Pontos Negativos e Desafios

- Loops Infinitos: Aumenta o risco de criar ciclos sem condições lógicas de saída, o que pode prender o leitor para sempre na mesma sequência. (Podemos fazer um algoritmo que procure por estes "ciclos infinitos" para dar um warning ao autor)
- Complexidade da Redução Transitiva: Em grafos com ciclos, a redução transitiva deixa de ser única e o cálculo torna-se demasiado pesado (NP-difícil). Identificar os atalhos passa a exigir aproximações mais complexas.
- Cálculo de Progressão: Num DAG clássico, a percentagem de história concluída é fácil de calcular. Com ciclos, a noção de progresso fica ambígua e obriga-nos a usar variáveis estritas (como o limite de cenas concluídas usado na simulação no twine).

2. O Modelo a Implementar: Lista de Adjacência Bidirecional

Nas nossas notas sobre Listas de Adjacência, destacámos que cada nó conhece apenas as suas ligações de saída. Mas como um dos requisitos centrais do Plot in a Pot é conseguir ver as escolhas que levam a cada cena, vamos ter de expandir este conceito. A solução que pensamos em implementar é uma Lista de Adjacência Bidirecional.

Em vez de usar uma Matriz de Adjacência, que desperdiça imensa memória e cresce ao quadrado ($O(V^2)$), esta lista ocupa apenas espaço proporcional ao número de cenas mais o número de escolhas ($O(V+E)$), mantendo o motor muito leve.

Para cada cena, o meu modelo de dados vai armazenar dois conjuntos distintos:

1. Saídas (Sucessores): As escolhas disponíveis para o leitor prosseguir.
2. Entradas (Predecessores): Um registo de todas as cenas e escolhas passadas que apontam para esta cena específica.

Estrutura Base em JSON

Para implementar isto no código, o formato do modelo será algo deste género:

JSON

```
{
  "cenas": [
    {
      "id": "cena_praca",
      "titulo": "A Praça Central",
      "conteudo": "O sol brilha no mercado.",
      "saidas": [
        {"texto_escolha": "Entrar na taberna", "destino": "cena_taberna"},
        {"texto_escolha": "Aguardar um momento", "destino": "cena_praca"}
      ],
      "entradas": ["cena_praca", "cena_taberna"]
    },
    {
      "id": "cena_taberna",
      "titulo": "Interior da Taberna",
      "conteudo": "O cheiro a vinho preenche o ar.",
      "saidas": [
        {"texto_escolha": "Voltar lá fora", "destino": "cena_praca"}
      ],
      "entradas": ["cena_praca"]
    }
  ]
}
```

3. Visualização e Interface de Acessibilidade

Como o Miguel é daltónico, não pode depender da cor das linhas para diferenciar o fluxo do grafo. Para garantir que percebe inequivocamente de onde vem e para onde vai, a interface tem de se basear em formas e padrões visuais claros.

A nossa estratégia de apresentação será:

- Quando uma cena é selecionada na área de trabalho gráfica, as linhas que representam as Entradas mudam para um padrão a tracejado e ficam mais grossas. As Saídas mantêm-se como linhas finas e contínuas com setas direcionadas a afastar-se da cena.

(Resetar estilo das setas -> Procurar as Setas que são entradas -> Meter o Estilo novo[Tracejado, Grosso, Cor de Mais contraste])

- Painel de Mapeamento Reverso: Além da vista gráfica de nós e linhas (inspirada no Twine), vamos adicionar uma barra lateral em texto estruturado (Toggle nas Opções da aplicação). Ao clicar numa cena, mostra uma tabela explícita e literal:
 - Vens de: [Cena X via "Escolha Y"]
 - Vais para: [Cena Z via "Escolha W"]

4. Próximos Passos para o Desenvolvimento

1. Motor de Nós Visuais: Vamos utilizar a inspiração visual do Twine para a grelha de criação de protótipos, mas montada sobre a arquitetura da Lista Bidirecional.
 2. Análise de Estrutura: Como o sistema agora permite ciclos e possíveis erros narrativos, vou implementar algoritmos que correm em segundo plano. O objetivo é alertar o utilizador sobre cenas inalcançáveis (com listas de Entradas vazias) e becos sem saída.
 3. Registo de Auditoria e Logger: Vou integrar a ideia do Logger de Auditoria para gravar o vetor de passos. Assim, enquanto testo a história dentro da aplicação, o motor guarda o caminho num registo de texto puro, facilitando a deteção de bloqueios na simulação.
- Algoritmo de Simulação, customizável ao autor para simulação das rotas dos jogadores

Mudanças

- Story Init -Cabeçalho com todas as variáveis para a simulação
- Sistema_motor - Processador de eventos "time-gated" ou eventos de múltiplos estágios
- Painel de Controlo - Header que mostra variáveis relevantes e que grava o caminho percorrido pelo jogador/simulação
- Falta implementar:
 - Pesos para a simulação
 - uma verdadeira máquina de turing que leva em relevância as variáveis existentes

conversão para simulação

1. Arquitetura: Motor de Estados (Fita de Dados)

Mudámos o Twine de uma simples "história de escolher caminhos" para uma **Máquina de Estados Finita**.

- **Passagem Central (Sistema_Motor):** Funciona como a CPU. Todas as transições narrativas passam por aqui para validar se o tempo esgotou ou se os objetivos foram atingidos.
- **Variáveis de Controlo (A Fita):**
 - \$tempo_praia: Limite de 6 rodadas (Time-box).
 - \$cenas_concluidas: Limite de 4 cenas de investigação (Progresso Narrativo).
 - \$log_caminho: Vetor (Array) que grava cada passo para auditoria pós-simulação.

2. Monitorização: O Logger de Auditoria

Implementámos um sistema de rastreio em tempo real que ignora a formatação visual para evitar erros de sistema.

- **Sanitização de Strings:** Criámos um script que converte sublinhados (_) em hífen (-) apenas no log. Isto impede que o Harlowe tente executar nomes de ficheiros como se fossem comandos, limpando os "erros roxos" do ecrã.
- **Header Dinâmico:** Um cabeçalho automático que mostra o **Local**, **Estado** e **Progresso** no topo de cada ecrã.

3. Inteligência: O Agente Lógico (Smart Walker)

Substituímos o clique aleatório por um algoritmo de **Exploração de Novidade**.

- **Memória Global:** O simulador agora guarda num Set de JavaScript todos os nomes de links que já foram clicados.
- **Poda de Árvore (Pruning):** O algoritmo filtra os botões disponíveis e **prioriza links que nunca visitou**.

4. Mecânicas de Jogo (Equilíbrio)

- **Teste Estendido (O Sonho):** Implementação de uma mecânica de "Cabo de Guerra" com 7 sucessos vs 3 falhas. A simulação revelou que esta DT é altamente punitiva (taxa de derrota elevada).
- **Blocks de Exploração:** Adição de *Flags* (etiquetas) que "trancam" locais já visitados no Povoado, forçando o avanço da história.

O que é:

- **Vértices e Arestas:** Um grafo é formado por vértices (nós) interligados por arestas. Num grafo direcionado, cada aresta possui uma orientação clara, fluindo de um vértice para um vértice o que é denotado matematicamente como $v \rightarrow w$. Na arquitetura do software, cada vértice representará um segmento da história (um capítulo ou cena), e cada aresta direcionada será uma decisão tomada pelo leitor.
- **Ausência de Ciclos:** A propriedade "acíclica" significa que é impossível iniciar num vértice seguir uma sequência de arestas direcionadas e terminar de volta no vértice. Esta lógica assegura que a progressão narrativa flui sempre em frente, mesmo que desemboque em finais diferentes ou que vários caminhos convirjam para um mesmo evento futuro. Na Imagem podemos ver que não importa como, começando em v não podemos voltar ao v .
- **Relação de Alcançabilidade:** Pode ser formalizada através de uma ordem parcial sobre os vértices do DAG. Diz-se que $v \leq w$ se existir um caminho direcionado de v até w . Em termos de implementação, isto permite ao software calcular rapidamente se um determinado final ou cena é alcançável a partir da cena atual.
- **Ordenação Topológica:** Um grafo é um DAG se e só se os seus vértices puderem ser alinhados numa sequência linear onde, para cada aresta $v \rightarrow w$, o vértice v aparece estritamente antes de w . Isto é particularmente útil para estruturar a interface gráfica da aplicação, permitindo desenhar a árvore de escolhas da história de forma limpa, da esquerda para a direita ou de cima para baixo.

Computação:

- **Ordenação Topológica (Topological Sorting):** Algoritmos clássicos (como o algoritmo de Kahn ou abordagens baseadas em pesquisa em profundidade) resolvem a ordenação topológica em tempo linear, onde V representa o número de vértices e E o número de arestas.

$$O(V + E)$$

- **Cálculo de Caminhos:** Em grafos arbitrários, encontrar o caminho mais longo é um problema complexo (NP-difícil). Contudo, num DAG, encontrar os caminhos mais curtos ou mais longos pode ser feito em tempo linear. Para o software, isto traduz-se na capacidade de calcular instantaneamente qual é a narrativa mais rápida para concluir a história ou qual é o percurso que explora o maior número de cenas.

Para a aplicação:

- **Estruturas Causais:** Os DAGs modelam relações de causa e efeito, assegurando que o tempo ou a progressão flui apenas numa direção. Um evento no futuro da narrativa só pode ser afetado pelas escolhas passadas que existam no seu caminho.

- **Histórico de Versões (Version History):** Sistemas de controlo de versões utilizam DAGs para rastrear alterações. No contexto da escrita, esta lógica pode ser usada para permitir ao autor criar versões alternativas do enredo, fundir (*merge*) diferentes arcos narrativos e manter um registo claro de que evento deu origem a que ramificação.
- **Grafos de Dependências:** Usados em sistemas de escalonamento (*scheduling*) para ditar que uma tarefa só pode ocorrer após a conclusão de outra. Numa narrativa interativa, uma cena pode ter uma dependência técnica (por exemplo, um vértice que exige que o jogador tenha recolhido uma chave num vértice anterior). O DAG garante que não existem dependências circulares impossíveis de resolver.

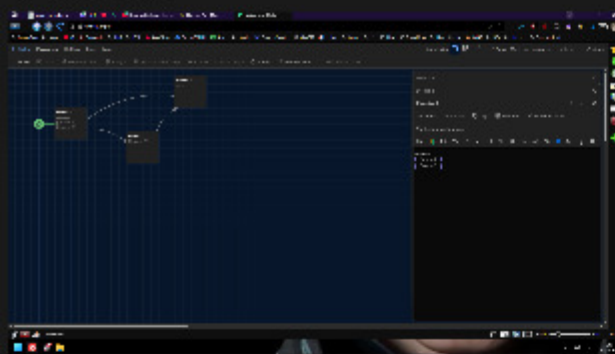
Todo:

MANDAR EMAIL À STORA SONIA

- Pesquisar softwares existentes para escrita de narrativas não lineares
 - pros e cons de cada
- Pesquisar software para mostrar graphs
- Tirar requisitos
- estudar cada método de implementação

Exemplo escrito para estudar com os stores na próxima reunião

Twine



O que é?

O **Twine** foi criado em 2009 por **Chris Klimas** como uma resposta direta à dificuldade de criar ficção interativa sem conhecimentos técnicos. A ferramenta baseia-se no conceito de **hypertext**, permitindo ao utilizador escrever histórias como uma rede de passagens ligadas por escolhas.

Tornou-se especialmente popular em comunidades independentes e experimentais, onde escritores, artistas e designers exploram temas pessoais ou narrativas não convencionais.

Exemplos

- Jogos: *Depression Quest*, *The Temple of No*
- Filmes: *Black Mirror: Bandersnatch* by **Charlie Brooker**



Pros

- Extremamente acessível (ideal para iniciantes)
- Visual e intuitivo (estrutura em grafo)
- Gratuito e open-source
- Muito rápido para a criação de protótipos ideais
- Forte adoção em comunidades criativas

Contras

- Estrutura torna-se confusa em projetos grandes
- Lógica e estado limitados
- Difícil integração com motores de jogo
- Não pensado para sistemas complexos de RPG

Ink



O que é?

O **Ink** foi desenvolvido pela **Inkle Studios** como uma solução interna para criar narrativas complexas em jogos.

A sua base conceptual é tratar a narrativa como código, permitindo ao autor definir lógica, variáveis e fluxo de forma programática.

Ao contrário do **Twine**, não privilegia a visualização, mas sim o controlo e a escalabilidade.

É amplamente usado por designers narrativos que trabalham em jogos com sistemas complexos e múltiplas consequências.

Exemplos

- Jogos: *80 days* by **Inkle Studio**



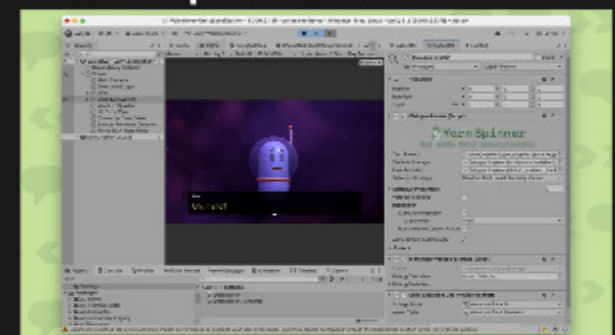
Pros

- Muito poderoso para lógica narrativa
- Permite controlo detalhado de estados e condições
- Escala bem para projetos grandes
- Ideal para narrativa sistémica

Contras

- Não visual (difícil perceber estrutura global)
- Requer pensamento técnico (quase programação)
- Pode ser difícil de manter sem disciplina
- Menos intuitivo para escritores tradicionais

Yarn Spinner



O que é?

O **Yarn Spinner** foi criado por **Jon Manning** e **Tim Nugent** com o objetivo de integrar narrativa diretamente no desenvolvimento de jogos.

A ferramenta combina uma estrutura baseada em nós com um sistema de variáveis e condições, funcionando como uma ponte entre escrita e programação.

É especialmente popular em jogos indie e projetos que usam motores como **Unity**.

Exemplos

- Jogos
 - *UNBEATABLE* by **D-CELL Games**
 - *Escape Academy* by **Colin Crew Games**
 - *Star Trek: Infection* by **Played With Fire**
- RPGs Indie



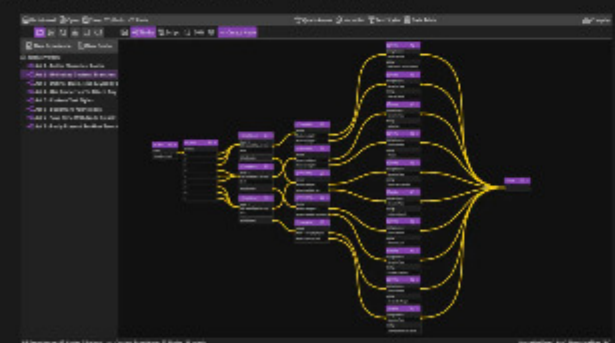
Pros

- Integração direta com motores de jogo
- Suporte a variáveis e estado
- Sintaxe relativamente simples
- Adequado para produção real
- Usado em muitos jogos comerciais

Contras

- Menos visual que ferramentas como **Twine**
- Depende de engine (**Unity**) para uso completo
- Pode tornar-se complexo em grande escala
- Não é ideal para planeamento abstrato

Narrative Flow



O que é?

O **Narrative Flow** é uma ferramenta relativamente recente criada para facilitar a construção de narrativas ramificadas, especialmente em jogos.

Surge como uma resposta às limitações de ferramentas como **Twine** e **Ink**, tentando combinar **visualização clara** + **lógica estruturada** + **integração com desenvolvimento**.

A ideia central é permitir ao utilizador construir histórias como fluxos organizados, mantendo controlo sobre variáveis e condições sem perder a visão global.

Exemplos

- Protótipos de RPGs narrativos



Pros

- Melhor organização que **Twine**
- Mais visual que **Ink**
- Suporte a lógica e estado
- Pensado para escalar melhor
- Ajuda a evitar inconsistências narrativas

Contras

- Ainda não amplamente testado (menos maturidade)
- Não impõe estrutura formal (podendo criar caos)
- Menos integração e comunidade que ferramentas grandes
- Programa não gratuito

Articy Draft



O que é?

O **Articy Draft** foi desenvolvido pela empresa **Articy Software GmbH** (ligada à **Nevego**) com o objetivo de resolver um problema específico da indústria: gerir narrativas complexas em equipas grandes.

A ferramenta funciona como uma combinação de editor visual e base de dados narrativa, permitindo organizar diálogos, quests, personagens e lógica num sistema unificado.

É amplamente utilizada em produção profissional, tanto em jogos indie como AAA.

Exemplos

- Jogos
 - *Cyberpunk 2077* by **CD Projekt RED**
 - *Tropico 6* by **Limbic Entertainment**



Pros

- Estrutura altamente organizada
- Visualização clara de fluxos narrativos
- Suporte a lógica, estados e condições
- Ferramenta profissional para equipas
- Escala bem para projetos grandes

Contras

- Complexo e pesado
- Curva de aprendizagem elevada
- Ferramenta paga
- Pode ser excessivo para projetos pequenos

- **Directed Graph (Grafo Direcionado):** Um **Directed Graph** é um grafo onde as ligações têm direção, ou seja, $A \rightarrow B \neq B \rightarrow A$. Na adjacency matrix, isto significa que a matriz não é simétrica:

$$M[i][j] \neq M[j][i]$$

- **Aplicação na narrativa:** Representa fluxo narrativo puro, onde cada escolha leva a uma direção específica sem implicar retorno.

- **Undirected Graph (Grafo Não Direcionado):** Um **Undirected Graph** representa relações bidirecionais, onde $A - B = B - A$. Na matriz, isto traduz-se numa estrutura simétrica:

$$M[i][j] = M[j][i]$$

- **Aplicação na narrativa:** É menos comum em storytelling puro, mas pode representar relações sociais ou conexões bidirecionais entre personagens ou estados.

- **Knowledge Graph (Grafo de Conhecimento):** Um **Knowledge Graph** representa relações semânticas mais complexas entre entidades, como “conhece”, “depende de” ou “controla”.

Neste caso, uma única matriz não é suficiente; é necessário usar múltiplas matrizes ou uma estrutura tridimensional:

Mrelation[i][j]

- **Aplicação na narrativa:** Permite modelar sistemas narrativos complexos baseados em conhecimento do jogador, estados e relações entre entidades.

- **Weighted Graph (Grafo Ponderado):** Um **Weighted Graph** atribui valores às ligações, em vez de apenas existência ou não existência. Esses valores podem representar custo, impacto, probabilidade ou importância. Na adjacency matrix:

$$M[i][j] = \textit{peso}$$

- **Aplicação na narrativa:** Permite modelar escolhas com impacto diferente, como custo emocional, dificuldade, ou relevância narrativa.